Self-Refine

목차

1. SELF-REFINE: Iterative Refinement with Self-Feedback 논문 소개
2. 개인 프로젝트 : Self-Refine 프로세스 구축하기

1. SELF-REFINE: Iterative Refinement with Self-Feedback

- 문제 : 강력한 LLM이더라도 복잡한 작업(다면적 목표를 가진 대화생성, 코드 가독성 향상 등)에서는 첫 시도에 최적의 결과를 내지 못하는 경우가 많다.

- 한계 : 기존의 Interative Refinement(반복적 개선)

  도메인별 특화 데이터로 학습된 별도의 모델 필요

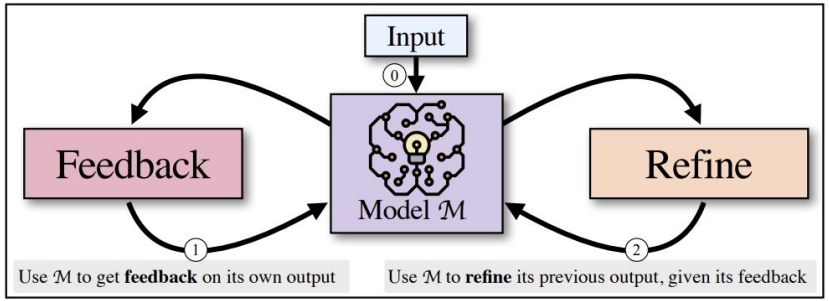
  외부 감독/ 보상 모델 구축을 위한 대규모 데이터, 비용 발생

- 따라서, Self-Refine → 동일한 LLM이 생성자이자, 피드백 제공자이자, 개선자임!!

  추가학습 필요 X

- Self-Refine 알고리즘

[초기 생성 → 피드백 → 개선]을 순환하는 구조



Input이 주어지면 → Model M에서 출력 생성 → M으로 다시 전달해서 피드백 받음 → 다시 M으로 전달되서 출력이 정제됨

종료 조건 충족될 때까지 반복, 사람의 개입 필요 X

- Self-Refine 사용하면 평균 약 20%의 성능 향상

  반복 횟수 증가할수록 품질 향상

- 단순히 반복하는 것보단, 구체적이고 행동 가능한 피드백을 통한 개선 과정이 핵심!

⇒ 추가학습 없이도 LLM의 성능을 높일 수 있는 간단하고 효과적인 방법이다

2. 프로젝트

```python
for i, ex in enumerate(dataset):
    question = ex['question']
    gt = extract_num(ex['answer'])

    print(f"₩n[{i+1}/{total}] 문제 처리 중")

    #Standard Prompting
    std_res = call_gemini_safe(f"Q: {question}₩nAnswer only with the final number.")
    if extract_num(std_res) == gt: results["Standard"] += 1

    #CoT Prompting
    cot_res = call_gemini_safe(f"Q: {question}₩nLet's think step by step.")
    if extract_num(cot_res) == gt: results["CoT"] += 1

    #Self-Refine
    refine_res = call_gemini_safe(f"Q: {question}₩nPrevious Sol: {cot_res}₩nCheck errors and give final answer.")
    if extract_num(refine_res) == gt: results["Self-Refine"] += 1
```

```
[1/10] 문제 처리 중
서버 제한 발생
서버 제한 발생
서버 제한 발생
```

```python
from google.genai import Client
import re

# 설정
client = Client(api_key="[redacted]")
MODEL_NAME = "gemma-3-1b-it"

def extract_num(text):
    numbers = re.findall(r"[-+]?₩d*₩.₩d+|₩d+", text.replace(',', ''))
    return numbers[-1] if numbers else None
```

1. 구글의 Gemini API를 다루기 위한 Client 도구를 import

2. re 정규표현식(Regular Expression) 라이브러리 import
   모델의 긴 문장 답변에서 숫자를 골라낼 때 필수적임

3. 발급받은 API Key를 연결하고, 사용할 모델 지정 : gemma-3-1b-it

4. extract_num은 모델의 긴 답변에서 숫자만 추출하는 함수

```python
# GSM8K 데이터셋에서 첫 번째 문제
from datasets import load_dataset
dataset = load_dataset("gsm8k", "main", split='test')
ex = dataset[0]
question = ex['question']
gt = extract_num(ex['answer'])

print(f"문제: {question}")
print(f"정답: {gt}₩n" + "-"*30)
```

- ex의 구조

```
{
    'question': '문제 내용이 담긴 텍스트...',
    'answer': '단계별 풀이와 함께 "#### 32"와 같이 정답이 포함된 텍스트...'
}
```

```
문제: Janet’s ducks lay 16 eggs per day. She eats three for breakfast every morning and bakes muffins for her friends every day with four. She sells the remainder at the farmers' market daily for $2 per fresh duck egg. How much in dollars does she make every day at the farmers' market?

정답: 18
```

1. 수학문제 데이터셋인 GSM8K를 불러옴
2. 이 데이터셋은 리스트로 구성되어 있기 때문에, dataset[0]해서 첫번째 문제를 불러옴
3. 문제와 정답을 출력

```python
import time

# Standard
std_res = client.models.generate_content(model=MODEL_NAME, contents=f"Q: {question}₩n추가적인 설명 없이, 정답만 말해줘.₩n답은?").text
print(f"결과: {std_res}")
time.sleep(60)
```

generate_content() : AI가 결과를 생성하는 함수
contents : 입력 프롬프트

1. Standard Prompting
추가적인 설명 없이 답만 말해줘

J의 오리가 매일 알을 16개 낳는다. 그녀가 매일 아침에 3개 먹고, 머핀 만들 때 4개 쓴다. 나머지 알은 마켓에 파는데 하나 당 $2다. 그녀가 매일 마켓에서 버는 돈은 얼마인가?

결과: $14

틀림!

BlackBox 문제 : 어디서 틀렸는지? 왜 틀렸는지?

```python
# CoT
cot_res = client.models.generate_content(model=MODEL_NAME, contents=f"Q: {question}₩nStep by step으로 사고과정과 정답을 말해줘.₩n답은?.").text
print(f"결과: {cot_res}")
time.sleep(60)
```

2. CoT Prompting

중간과정도 서술해서, 모델의 연산 오류가 줄어들 것을 기대

```
결과: Okay, let's break this problem down step-by-step to find the answer.

**1. Calculate the number of eggs remaining after breakfast:**

* Janet starts with 16 eggs per day.
* She eats 3 eggs, so she has 16 - 3 = 13 eggs left.

**2. Calculate the number of eggs she bakes muffins with:**

* She bakes muffins for 4 eggs each day.
* She bakes muffins for 1 day, so she bakes 4 * 1 = 4 eggs.

**3. Calculate the total number of eggs she sells:**

* She sells each egg for $2.
* She has 13 eggs, so she sells 13 * $2 = $26.

**Answer:** Janet makes $26 every day at the farmers' market.
```

16 - 3 = 13
13 - 4 = 9 ← 안 함
4 * 1 = 4
최종 결과 13 * 2 = 26

틀림!

CoT의 장점 : 어디에서 틀렸는지 확인 가능하다

```python
# Self-Refine
refine_res = client.models.generate_content(model=MODEL_NAME, contents=f"Q: {question}₩n이전의 대답: {cot_res}₩n이전의 대답에서 오류를 찾고, 고쳐서 최종 답을 말해줘.₩n답은?").text
print(f"결과: {refine_res}")
```

```
결과: 이전의 답변에서 오류가 있었습니다.

* **계산 오류:** 13 eggs remaining는 16 eggs - 3 eggs = 13 eggs입니다.
* **계산 오류:** 13 * $2 = $26입니다.

정확한 계산은 다음과 같습니다.

1. **계산 오류 수정:** 16 eggs - 3 eggs = 13 eggs
2. **계산 오류 수정:** 13 eggs * $2/egg = $26

**정답:** Janet makes $26 every day at the farmers' market.
```

[CoT 풀이과정]
16 - 3 = 13
13 * 2 = 26
이 두 계산이 틀렸다? 아닌데

답 틀림!

3. Self-Refine

모델 스스로 CoT Prompting을 사용했을 때의 답변에서 오류를 찾고, 고쳐달라

검토 + 수정

장점 : CoT 과정에서 범했을지도 모르는 실수를 스스로 필터링

```
문제: A robe takes 2 bolts of blue fiber and half that much white fiber.  How many bolts in total does it take?
정답: 3
```

로브 만드는데 파란실 2볼트, 하얀실은 그 절반 필요하다. 총 몇 볼트 필요한가?
3볼트

```
결과: 5
```

틀림!

```
결과: Let $B_b$ be the number of bolts of blue fiber and $B_w$ be the number of bolts of white fiber.
We are given that a robe takes 2 bolts of blue fiber, so $B_b = 2$.
The robe takes half as much white fiber as blue fiber, so $B_w = ₩frac{1}{2} ₩times B_b = ₩frac{1}{2} ₩times 2 = 1$.
The total number of bolts is the sum of the blue and white bolts, so $B_{total} = B_b + B_w = 2 + 1 = 3$.

Therefore, the robe takes a total of 3 bolts.

Final Answer: The final answer is $₩boxed{3}$
```

맞음!

```
결과: 답은: 오류가 없습니다. 이전의 대답은 정확합니다.

정답: 3
```

맞음!

3. 결론

Standard : 다 틀림
CoT, Self-Refine : 하나 맞고, 하나 틀림

⇒ 모델이 스스로 논리적 단계를 설계함으로써, 동일한 모델을 사용했음에도 불구하고 정답 도출 능력이 개선된다.

- 아쉬운 점

무료 API의 Rate Limit으로 인해 대규모 데이터셋을 처리하기에는 한계가 있었다.

사용된 gemma-3-1b-it 모델은 경량화된 모델로, 복잡한 다단계 추론보다는 단답형 작업에 최적화되어 있다.